ANIMUSCORE TECHNOLOGIES PRIVATE LIMITED

AnimusCore Execution & Telemetry Engine

Institutional Architecture & Hardware Performance Whitepaper v2.4.0

Organization: AnimusCore Technologies Private Limited
Founder & Lead Systems Architect: Alakshendra Roy
Audience: Prop-desk CTOs, execution-infrastructure leads, low-latency C++ reviewers
Classification: Commercial Technical Specification
Supersedes: v1.1.0

CONTENTS

1. Headline Figures
2. 1. Executive Summary
3. 2. Measurement Discipline
4. 3. Mechanical Sympathy & Cache Topology
5. 4. Dual-Buffer Topologies
6. 5. Hardware Timing
7. 6. Cross-Language Zero-Copy Interop
8. 7. Memory & Heap Discipline
9. 8. Measured Results
10. 9. Platform Scope
11. 10. Delivery & Engagement Model
12. 11. Limits & Non-Claims
13. 12. Reproducing These Results
14. Document Control

Headline Figures

Every figure below is a measurement with a stated definition, platform and run count. Section 2 defines the latency terms; Section 8 gives ranges and provenance. None of these is a guarantee. They are the readings the Proof-of-Performance kit reproduces on your hardware.

Quantity	Value	What it is
Push path, single core	**9.5–9.9 ns median**, 13.6–17.4 ns p99	`SpscQueue<TelemetryPayload>` enqueue + dequeue on one core (WSL2 guest, CPU 3, no load)
Producer push cost	**~21 ns median**	Producer-side `push` P50 across a 153-run soak, one machine, cores pinned, not kernel-isolated
Lossless SPSC sustained flood	**17.8–27.6 M events/s** (median 21.5 M), 30 runs	Producer and consumer on separate physical P-cores, bare-metal host. Supersedes the 8.21 M/s figure of v1.1.0
Cross-core hand-off, SPSC	**p50 103–185 ns** (median 153), p99 158–289 ns	Separate physical P-cores, 30 runs, two compilers. Tail: p99.9 ≈ 2.4–2.6 µs (scheduler noise)
Paced tick-to-telemetry	**p50 ~115–125 ns**	4-hour paced soak, open-loop offered load, stamps taken at the scheduled send time (WSL2 guest)
Python boundary	**~34.5 ns/event** amortized `drain()` ; 215–335 ns per `slots()` view creation	nanobind zero-copy ndarray over the shared ring. Not a turnaround. See Section 6
Heap allocation, hot path	**0 calls, 0 bytes** after initialization	Checked by a glibc allocator interposer (29 entry points) over 10 M-iteration windows and a 4-hour soak
Timestamping	Invariant TSC (`lfence; rdtsc; lfence`)	User-mode read, no syscall

1. Executive Summary

AnimusCore is a C++17 header-only telemetry transport and execution-event engine for latency-critical Linux hosts, with a zero-copy Python bridge. It provides three shared-memory ring topologies, a cycle-accurate timing layer, and a verification harness that checks the engine's claims on the customer's own hardware.

The engineering objective is narrow. Between "an event exists in memory" and "a consumer, in C++ or Python, can read it", AnimusCore removes every avoidable cost: heap allocation, serialization, kernel transitions, lock acquisition and cache-line contention. What remains is a bounded number of cache-coherency transactions per event, and that cost is determined by the host. The engine cannot reduce it further.

This document specifies the mechanism, states what was measured and under which conditions, and lists what was not measured. A claim that is not backed by a reproducible command in the verification kit does not appear here.

---

2. Measurement Discipline

Latency figures from lock-free queues are routinely misreported by mixing four different quantities. This document keeps them separate.

Term	Definition	Where it is used here
Push cost	Time the producer spends inside `push` / `try_push`. Producer-local.	21 ns producer push
Single-core enqueue + dequeue	Both operations on one core. Measures instruction path and L1 behaviour, not coherency traffic.	9.5–9.9 ns. This is Proof-of-Performance criterion (b)
Cross-core hand-off	Stamp at producer, read at consumer on a different physical core. Includes cache-line transfer.	103–185 ns p50
Amortized drain cost	Total `drain()` time divided by events in the batch.	34.5 ns/event Python bridge

Rules this document follows

1. **Paced, not flat-out.** Latency is quoted from open-loop paced runs. Under flat-out producers, a bounded ring fills and "latency" becomes queue residence time. In the audit harness the same rings reported a flat-out p50 of 12.4 ms against 90–132 ns when spaced. Burst-queue medians are not quoted as latency.
2. **Physical cores are verified, not assumed.** Logical processors 0 and 1 on a hyperthreaded part are sibling threads of one core. Siblings share L1/L2, so a "cross-core" run on them reports 2–4× lower latency than a real cross-core hand-off (p50 41–58 ns on siblings against 103–185 ns on separate P-cores, same machine). All cross-core figures here were taken on topology-verified separate physical cores. Sibling-pair figures are labelled as such.
3. **Clock placement is stated.** Stamping per event versus per span changed p50 by up to 2.9× in the audit harness. The stamp point is part of the result.
4. **Virtualized runs are labelled.** Numbers from a WSL2 guest include hypervisor effects. Guest vCPU numbering says nothing about physical placement there. They are reported as guest numbers and are not presented as bare-metal results.

---

3. Mechanical Sympathy & Cache Topology

3.1 Cache-line discipline

Every record that crosses a producer/consumer boundary is exactly one cache line, enforced at compile time:

```
struct alignas(64) TelemetryPayload { /* 64 bytes: sequence, dispatch_tsc, ids, value, reserved */ };
static_assert(sizeof(TelemetryPayload) == 64, "one cache line");
static_assert(alignof(TelemetryPayload) == 64, "line aligned");
```

A change that grows the record past 64 bytes fails the build rather than a benchmark months later.

3.2 Cursor isolation

Ring control state is split so that no two threads write the same line:

- The producer's write cursor, the consumer's read cursor and any shared flags each occupy their own `alignas(64)` region.
- Each side keeps a **cached copy of the peer's cursor** and re-reads the shared cursor only when its cached view says the ring is full (producer) or empty (consumer). In steady state a push touches the slot line and the producer's own cursor line. The consumer's line is not touched.
- Publication uses `memory_order_release`, observation uses `memory_order_acquire`. Sequential consistency is not used. In the audit harness a `seq_cst` publish cost +26 ns p50 (140 → 167 ns) and cut flood throughput by 2–4×.

3.3 Measured effect of line isolation

A direct A/B: two atomic counters incremented by two threads on distinct cores, once sharing a line and once with `alignas(64)` separation.

Configuration	Combined throughput
Shared line (false sharing)	~162–164 M ops/s
`alignas(64)` separation	~214–215 M ops/s
Ratio	~1.3×

This A/B isolates the effect of invalidation traffic. It is not the ring's throughput.

3.4 What alignment does not do

- It does not remove the cost of the one cache-line transfer a cross-core hand-off requires. That transfer is the floor of the 103–185 ns cross-core figure.
- It does not control adjacent-line prefetch. An audit probe on Raptor Cove found no penalty from head and tail sharing a 128-byte sector, but that is one microarchitecture.
- It does not pin threads. Core placement is the integrator's responsibility; `thread_affinity.hpp` provides the primitive, and the harness prints the placement it used.

---

4. Dual-Buffer Topologies

Two delivery contracts need two different data structures. The engine does not use one ring with a mode switch.

4.1 `SpscQueue<T>`: lossless, wait-free, 1 → 1

- **Contract:** every event pushed is delivered once, in order. If the ring is full, `try_push` returns failure and the caller decides. There is no silent overwrite.
- **Progress:** wait-free for both sides. No CAS, no retry loop in the common path.
- **Use:** order-flow events, execution reports, anything where a gap is a correctness failure.
- **Evidence:** 138.47 G records delivered exactly in the 4-hour soak with zero sequence loss (Section 8.4).

4.2 `BroadcastRing<T>`: lossy, non-blocking, 1 writer → N readers

- **Contract:** the writer never waits and never observes reader state. Each reader holds its own cursor. A reader that falls more than one lap behind detects the overwrite from the slot sequence and resynchronizes; the number of lost records is reported.
- **Use:** market-depth fan-out, book snapshots, monitoring taps. Stale data is worthless and a slow reader must not stall the producer.
- **Evidence:** median hand-off ~114 ns in the paced soak. 387,652 records dropped across the 4-hour run. The drops are the designed behaviour, are counted, and are never silent.

4.3 `ShmRingMpmc<T>`: cross-process, lock-free, many → many

A per-slot-sequence ring (Vyukov-style) over POSIX shared memory for cases that need several producers or consumers. It fails cleanly (error return, not a fault) on attach to a stale or mismatched segment. Measured sustained throughput is **6.7–7.3 M events/s** at 8 producers / 8 consumers. This is lower than SPSC by design, because producers contend on a CAS. The topology that fits the contract should be selected, and MPMC is not a faster SPSC.

4.4 Selection

Need	Topology
Lossless, exactly one producer and one consumer	`SpscQueue`
One producer, many readers, freshness over completeness	`BroadcastRing`
Several producers or consumers, possibly cross-process	`ShmRingMpmc`

---

5. Hardware Timing

- **Source:** the invariant TSC on x86-64, read with `RDTSC`. The generic counter is the AArch64 equivalent. A read is a user-mode instruction with no syscall and no vDSO call.
- **Calibration:** done once at startup against a monotonic reference, and verified to be invariant, so frequency scaling does not distort results. The calibrated rate on the reference host is 2.4192 GHz.
- **Ordering:** measurement stamps use `lfence; rdtsc; lfence` (x86-64) or `isb` + counter read (AArch64) so out-of-order execution cannot move the read across the work being timed. A bare `rdtsc` is available for cases where the ordering cost is unwanted, and it is labelled as such.
- **Virtualization:** inside a guest, the TSC is only as stable as the hypervisor's offsetting and scaling make it. On the WSL2 reference guest the clocksource is `tsc` and the cores read consistently. A deployment on a different hypervisor must re-verify this, and the verification kit includes that check.
- **Cost of the stamp itself** is included in every measured figure. Subtracting it is not done.

---

6. Cross-Language Zero-Copy Interop

6.1 Mechanism

The Python SDK does not serialize, copy or cross a process boundary to read events. A nanobind module exposes the shared ring's slot memory as a NumPy `ndarray` with a structured dtype matching the 64-byte record. The array aliases the ring memory, with no owner and no copy. A strategy process reads the same bytes the engine wrote.

Two implementation facts matter for reviewers:

- nanobind returns an ndarray by **copying** under its default return policy. The bridge binds with `rv_policy::reference_internal`, and the test suite proves aliasing by comparing buffer addresses rather than assuming it.
- The module is built against the platform's native shared-memory semantics, with no OS-specific cryptography or IPC dependency in the data path.

6.2 What the numbers mean

Figure	Quantity	Value
Amortized `drain()`	Total drain time ÷ events in batch	~34.5 ns/event (repeat runs 22–66 ns/event, freeze-verification 36.8–42.7)
`slots()` view creation	Cost of one call that builds the ndarray view	215–335 ns per call (Python 3.14, NumPy 2.5)

The 34.5 ns figure is a batch-amortized throughput cost, not a turnaround latency. A single event read through a fresh `slots()` call costs the view-creation figure plus the read. Strategies that need low per-event cost in Python should hold the view and drain in batches. Per-event Python code on top of the view runs at interpreter speed, which this engine does not change.

---

7. Memory & Heap Discipline

- **Allocation model:** all rings, payload arrays and profiler buffers are sized and placed at initialization. The hot path has no `new`, `malloc`, container growth or exception allocation.
- **Verification is dynamic, not by inspection.** A glibc interposer covers 29 allocator entry points, including `valloc` / `pvalloc`, `aligned_alloc`, `posix_memalign` and `realloc`, with a thread-local enforcement flag that is set only around the measured hot path. Each entry point has a negative control (an injected allocation that must be caught, including a forked process that must abort), so a silent interposer failure is detected.
- **Measured:** 10 M-iteration windows on SPSC, MPMC and the profiler report 0 calls. The 4-hour soak reports **0 heap calls and 0 page faults** inside the measured window.
- **Memory residency:** ring memory is pre-faulted. `MAP_POPULATE` pre-faults but does not lock pages, so production deployments should additionally `mlock` the region. The kit's page-fault counter shows whether you need to.

---

8. Measured Results

8.1 Reference platform

Intel Core i7-14650HX (8 P-cores with SMT + 8 E-cores), GCC 15.2 and Clang 22, `-O3 -march=native`, C++17. The P/E split was confirmed with a per-core pinned-loop probe, not taken from the spec sheet. Two environments produced the data:

- **Bare-metal host** (Windows 11 build, topology verified with `GetLogicalProcessorInformationEx`): cross-core and flood figures.
- **WSL2 Ubuntu guest** (glibc 2.43, 12 cores × 2 threads exposed): single-core profile, zero-alloc audit and the 4-hour soak.

8.2 Cross-core SPSC (separate physical P-cores, 30 runs, MinGW g++ and MSVC)

Metric	Range	Median
p50 hand-off	103–185 ns	153 ns
p99 hand-off	158–289 ns	220 ns
p99.9	~2.4–2.6 µs	scheduler noise on a general-purpose host
Sustained flood	17.8–27.6 M/s	21.5 M/s

Same machine, SMT-sibling pair (not cross-core): p50 41–58 ns, p99 53–79 ns, 46.6–48.5 M/s.

8.3 Single-core push path (Proof-of-Performance criterion (b))

`SpscQueue<TelemetryPayload>`, pinned, no load, WSL2 guest, CPU 3: **median 9.5–9.9 ns, p99 13.6–17.4 ns.** p99 varied run to run. These are first readings on the reference guest and not customer hardware.

8.4 Four-hour paced soak (14,400 s window, WSL2 guest, 50 cycles / 298 phases)

Item	Result
Records delivered	138.47 G, exactly, no sequence loss on lossless rings
Heap calls / page faults in window	0 / 0
Paced median hand-off	SPSC 122 ns, Broadcast 114 ns, MPMC 211 ns
SPSC flood / MPMC flood	67.3 M/s / 6.67 M/s
Broadcast drops	387,652, counted, lossy by design
Host stall floor	1.61 % of vCPU time lost to stalls longer than 500 ns
Negative controls	Alloc, drop, corruption, stall and abort injections were all caught

The soak's tail behaviour tracks the host's own stalls: the measured tail never exceeded the independent stall probe's prediction (max 1.14×). The tail therefore reflects the hypervisor and OS, not the ring. A kernel-isolated bare-metal core is expected to do better. That has not been measured, and no figure for it is given here.

8.5 Cross-checks

- Build matrix: warning-clean under `-Wall -Wextra -Wpedantic -Werror` on GCC 13–16 and Clang 19 and 22.
- Sanitizers: ASan + UBSan clean. The MPMC protocol passes ThreadSanitizer, with 0 reports on an 8-producer/8-consumer 8 M-frame soak.

9. Platform Scope

- **Reference platform:** Linux x86-64, glibc, C++17. The data path uses POSIX shared memory (`shm_open` + `mmap`) and the standard library, with no third-party runtime dependencies.
- **Distribution form:** header-only C++17 core (`include/animus/*.hpp`), foldable into one self-contained header (`scripts/make_single_header.py`, which inlines the headers in dependency order and refuses output that does not compile as a translation unit), plus a curated C-ABI shared library for non-C++ callers and the nanobind bridge.
- **AArch64:** cycle timing is implemented (generic counter). Ring protocols are portable C++17 atomics. Full verification on AArch64 hardware has not been done, and it is reported as untested for production latency.
- **Windows:** a parity build exists and supplied some figures above, but it is not the delivery target of this document.
- **Not part of the hot path:** no entitlement, licensing or cryptographic check executes on the data path.

---

10. Delivery & Engagement Model

Engagement is asynchronous and artifact-driven. There are no scheduled calls, no on-call rotation and no screen-share requirement.

1. **Sanitized evaluation build.** A curated shared library with a fixed 13-function C ABI and an allowlist of stages. It exposes the interface and none of the engine internals beyond it. It ships with a SHA-256 checksum.
2. **Isolated sandbox harness.** A drop-in quickstart and soak harness that runs on an isolated node with no network access and no connection to live data. It prints core placement, TSC calibration, allocator-audit status and page-fault counts alongside every result, so an invalid run is visible.
3. **Proof-of-Performance (zero cost, 14–30 days).** Three pass/fail criteria, measured by the verification kit on your hardware:
   - (a) zero heap allocations over 10 M cycles;
   - (b) single-core SPSC enqueue + dequeue below **15 ns p50 and 25 ns p99**;
   - (c) `alignas(64)` layout with no false sharing (layout assertions plus `perf c2c` where counters are available).

Criterion (b) is deliberately a single-core push-path measurement. Cross-core hand-off is reported alongside it but is not a pass criterion, because it is a property of your topology.

1. **Written advisory.** Integration review, diffs and patches go through your issue tracker or a git remote. Response commitment is best-efforts within 24–48 business hours, in writing. The commercial terms are in the Institutional Enterprise Order Form and Master Agreement, not in this document.

Contact is by written channel via the repository's issue tracker or the founder's LinkedIn profile.

---

11. Limits & Non-Claims

These items were not measured or are not claimed:

- **No guaranteed latency.** Figures are measurements under stated conditions. The engine ships no bound, and the order form's Exhibit 1 states no performance warranty.

- **No tick-to-trade claim.** Measurements cover transport and timestamping inside the host. They exclude NIC, kernel-bypass stack, wire, exchange gateway and strategy compute. Kernel-bypass paths (AF_XDP, ef_vi, DPDK) have not been measured.
- **Not measured:** NUMA cross-socket behaviour, real SMI interference, live PMU/MSR paths (WSL2 exposes no PMU, so `perf c2c` has not run on real counters), bare-metal isolated-core tails.
- **Tail latency is host-dominated.** p99.9 and above on a general-purpose OS reflect scheduler and hypervisor behaviour. Without `isolcpus` / `nohz_full` /IRQ steering, expect microsecond-scale tails.
- **Hyperthread siblings flatter results.** See Section 2, rule 2. Any vendor figure that does not state core topology should be treated accordingly.
- **Split-lock and platform hazards:** an atomic spanning two cache lines caused a measured ~25 µs split-lock stall in the audit harness. The layout assertions prevent that in the shipped types, but they must be preserved by integrators who extend the payload.
- **Python:** the zero-copy view removes copies and serialization. It does not make interpreter-level per-event code fast.

---

12. Reproducing These Results

All commands run from the evaluation kit on an isolated Linux node.

```
make verify                      # build matrix, layout assertions, unit + negative controls
make -C lowlat check             # zero-alloc audit, TSC profiler, layout, bridge aliasing test
make -C lowlat pop               # Proof-of-Performance criteria on this host (machine-readable
POP_RESULT lines)
./run_demo.sh <prod-core> <cons-core>   # cross-core hand-off + flood; pass cores on DIFFERENT physical
cores
```

Before trusting any run, confirm that the harness reports: distinct physical cores, calibrated invariant TSC, allocator-audit enabled and 0 page faults. A run that cannot report these is marked INVALID rather than NOT MET.

Document Control

Field	Value
Version	2.4.0
Replaces	1.1.0 (Windows CNG licensing; 8.21 M events/s; P99.9 < 1.85 µs). The v1.1.0 headline metrics are withdrawn, not carried forward.
Author	Alakshendra Roy, Founder & Lead Systems Architect
Organization	AnimusCore Technologies Private Limited
Evidence	`benchmarks/reports/telemetry_rerun_2026-10-04/` , the 4-hour soak report (2026-10-12), `lowlat/` verification kit